```
C:\Users\My>arp -a

Interface: 192.168.0.105 --- 0x3
  Internet Address      Physical Address      Type
  192.168.0.1           b0-a7-b9-ad-87-e2     dynamic
  192.168.0.106         fa-2f-2a-9a-0a-2a     dynamic
  192.168.0.255         ff-ff-ff-ff-ff-ff     static
  224.0.0.22            01-00-5e-00-00-16     static
  224.0.0.251           01-00-5e-00-00-fb     static
  224.0.0.252           01-00-5e-00-00-fc     static
  239.255 255.250       01-00-5e-7f-ff-fa     static
  255.255.255.255       ff-ff-ff-ff-ff-ff     static

Interface: 172.29.208.1 --- 0x28
  Internet Address      Physical Address      Type
  172.29.217.16         00-15-5d-2f-9f-93     dynamic
  172.29.223.255        ff-ff-ff-ff-ff-ff     static
  224.0.0.22            01-00-5e-00-00-16     static
  224.0.0.251           01-00-5e-00-00-fb     static
  239.255.255.250       01-00-5e-7f-ff-fa     static
  255.255.255.255       ff-ff-ff-ff-ff-ff     static
```

```
\My>tracert google.com

route to google.com [142.250.77.142]
aximum of 30 hops:

1 ms     9 ms    15 ms  192.168.0.1
5 ms     2 ms     2 ms  10.208.0.1
*        *        *     Request timed out.
*        *        *     Request timed out.
5 ms     2 ms     2 ms  72.14.212.80
3 ms     3 ms     2 ms  192.178.120.189
6 ms     5 ms     3 ms  142.251.55.61
0 ms     2 ms     4 ms  maa05s16-in-f14.1e

mplete.
```

EXP-1

BASIC NETWORKING COMMANDS IN LINUX AND WINDOWS

Aim:

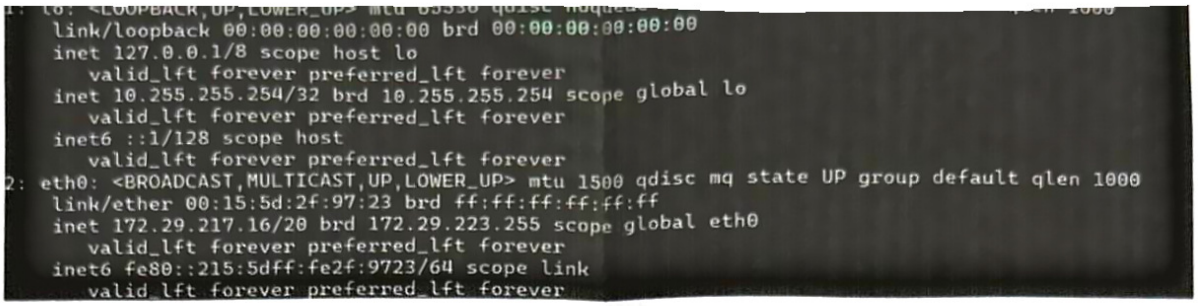
To study and execute basic networking commands available in Linux and windows operating systems For diagnosing and troubleshooting network issues

Commands and output:

Command	purpose	Sample output
arp-a	Display ARP cache	shows interface, internet address, physical address, type
hostname	Displays the computer name	Desktop-pc01

Linux Network Config (Using nmcli)

The Lab also demonstrates configuration IPV4/IPV6 settings using nmcli, activating due connection profile, verifying IP addresses using ip address, checking routes with IP routes, viewing DNS, settings and testing Connectivity using ping



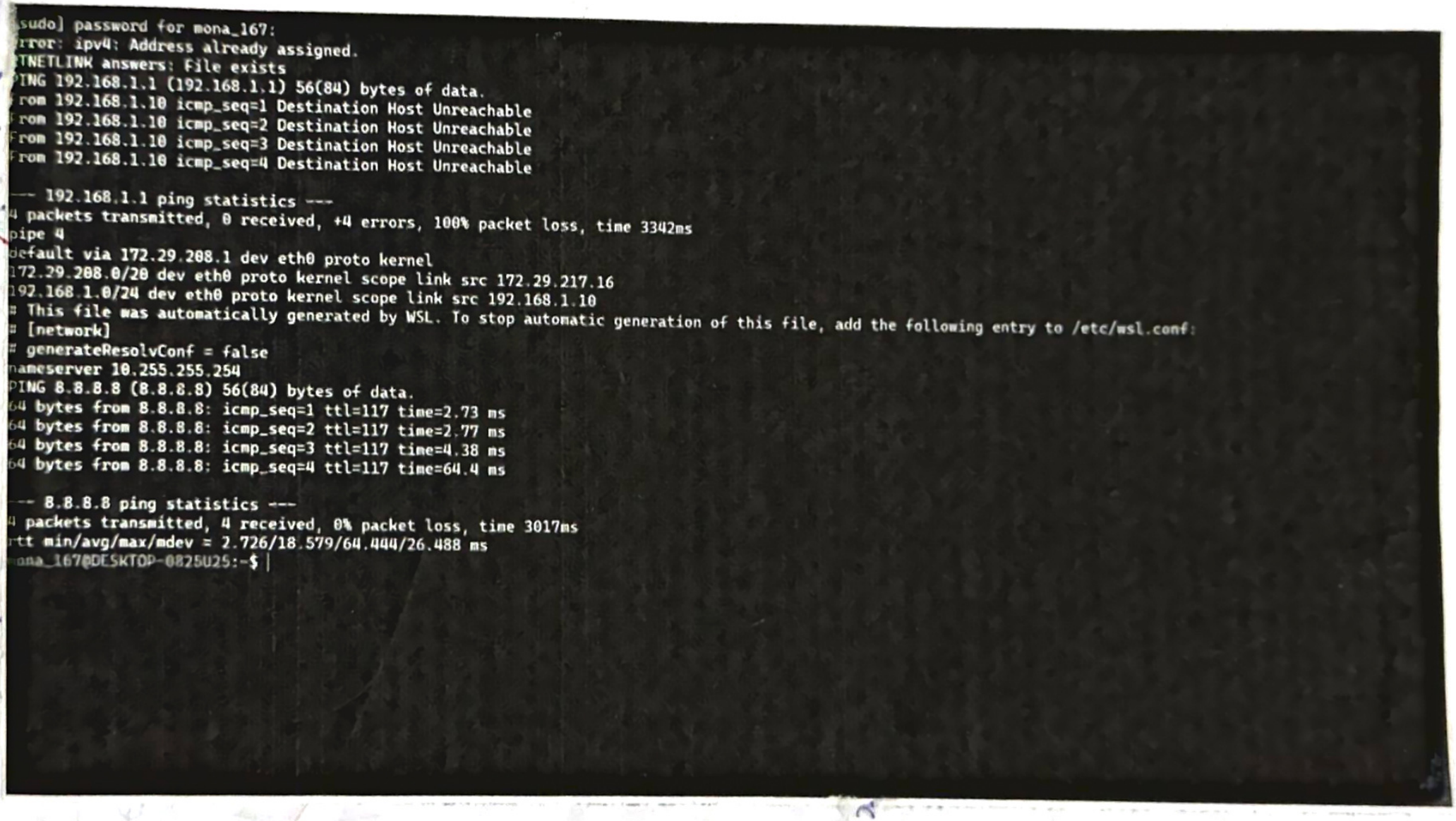

```
ona_167@DESKTOP-0825U25:~$ ip link show
: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOW
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc mq state
    link/ether 00:15:5d:2f:97:23 brd ff:ff:ff:ff:ff:ff
```

```
ona_167@DESKTOP-0825U25:~$ curl google.com
HTML><HEAD><meta http-equiv="content-type" content="text/html;charset=utf-8">
TITLE>301 Moved</TITLE></HEAD><BODY>
H1>301 Moved</H1>
he document has moved
A HREF="http://www.google.com/">here</A>.
/BODY></HTML>
```

```
ona_167@DESKTOP-0825U25:~$ ip route
efault via 172.29.208.1 dev eth0 proto kernel
72.29.208.0/20 dev eth0 proto kernel scope link src 172.29.217.16
```

Result:

The various networking commands in windows and Linux were exected successfully and the network config, connectivity, routing, were verified.


sudo] password for mona_167:
rror: ipv4: Address already assigned.
TNETLINK answers: File exists
PING 192.168.1.1 (192.168.1.1) 56(84) bytes of data.
rom 192.168.1.10 icmp_seq=1 Destination Host Unreachable
rom 192.168.1.10 icmp_seq=2 Destination Host Unreachable
From 192.168.1.10 icmp_seq=3 Destination Host Unreachable
From 192.168.1.10 icmp_seq=4 Destination Host Unreachable
-- 192.168.1.1 ping statistics ---
4 packets transmitted, 0 received, +4 errors, 100% packet loss, time 3342ms
pipe 4
default via 172.29.208.1 dev eth0 proto kernel
172.29.208.0/20 dev eth0 proto kernel scope link src 172.29.217.16
192.168.1.0/24 dev eth0 proto kernel scope link src 192.168.1.10
This file was automatically generated by WSL. To stop automatic generation of this file, add the following entry to /etc/wsl.conf:
[network]
generateResolvConf = false
nameserver 10.255.255.254
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=117 time=2.73 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=117 time=2.77 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=117 time=4.38 ms
64 bytes from 8.8.8.8: icmp_seq=4 ttl=117 time=64.4 ms
-- 8.8.8.8 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3017ms
rtt min/avg/max/mdev = 2.726/18.579/64.444/26.488 ms
ona_167@DESKTOP-0825U25:~$

EXP-2

LEARNING AND ASSIGNMENT OF IP ADDRESS MANUALLY TO COMPUTER

Aim:

To understand IP addressing scheme and manually assign static IP address to computer on a network.

Theory:

An ip address is a unique numerical label assigned to each device connected to a computer network.

IPV4 address are 32-bit number written in dotted-decimal notation.

(Eg 192.168.1.10)

IPV6 user 128 bit address

Code:

<1 Bash)

Sudo ip addr add 192.168.1.10.124 are echo

Sudo ip route add defant via 192.168.1.10

ip addr show echo

ping C4 192.168.1.1

Sudo nueli con mod" wived connection"

IPV4 address 192.168.1.10

Sudo nmcli COU mod "wired connection"

IPV4 getway 192.108.1.1

Sudo nmcli Con mod "wired connection"

IPV4 metred mannal

Sudo nmcli con mod "wired connection"
IPV4 .dns "8.8.8.8"

sudo nmcli con up "wired connection"

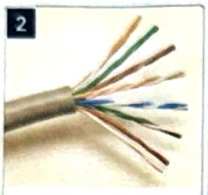
Result:

Static IP address were successfully assigned to compute connectivity was verified using ping commands between host an de same subset

Straight-Through Cable (568B – 568B)



Cut the required length of UTP cable.



Strip approximately 2.5 cm of outer jacket.



Arrange the wires in TIA/EIA 568B order.



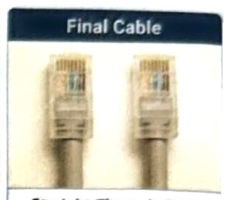
Trim all wires to equal length (~1.5 cm from the jacket).



Insert wires into RJ45 connector, ensure each wire reaches front.



Crimp firmly using the crimping tool.

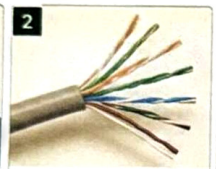


Straight-Through Cable (568B – 568B)

Cross-Over Cable (568B – 568A)



Cut the required length of UTP cable.



Strip approximately 2.5 cm of outer jacket.



Arrange the wires in TIA/EIA 568B order (End 1).



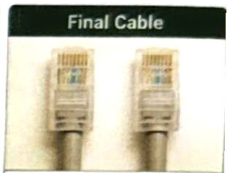
Insert into RJ45 connector and crimp (End 1).



Arrange the wires in TIA/EIA 568A order (End 2).



Insert into RJ45 connector and crimp (End 2).



Cross-Over Cable (568B – 568A)

Cable Tester Results

Straight-Through Cable (568B – 568B)



LED Sequence	
1	✓
2	✓
3	✓
4	✓
5	✓
6	✓
7	✓
8	✓
G	✓

All 8 pins + G show proper continuity.

Cross-Over Cable (568B – 568A)



LED Sequence	
1	✓
2	✓
3	✓
4	✓
5	✓
6	✓
7	✓
8	✓
G	✓

All 8 pins + G show proper continuity.

Wiring Standards (Pinout)

TIA/EIA 568B (most common)								
Pin No.	1	2	3	4	5	6	7	8
Color	Orange-White	Orange	Green-White	Blue	Blue-White	Green	Brown-White	Brown

TIA/EIA 568A								
Pin No.	1	2	3	4	5	6	7	8
Color	Green-White	Green	Orange-White	Blue	Blue-White	Orange	Brown-White	Brown

Straight-Through: 568B at both ends (PC ↔ Switch)
Cross-Over: 568B at one end and 568A at the other (PC ↔ PC)

EXP-3

STUDY OF NETWORK CABLES AND CRIMPING WITH RJ45 CONNECTION

Aim:

To study different types of network cables and to prepare a straight through and cross-over Ethernet cable by crimping RJ45 connection

Theory:

1. Twisted pair cable:
   UTP and STP
   Used in LANS.
   Categories:
   → Cat 5
   → Cat 5e
   → Cat 6
   → Cat 6a

2. Coaxial Cable:
   Used in cable TV and other internet (10 Base2, 10 Base5)
   High shielding

3. Fibre optic cable:
   Used in cable TV and other very high band width

RJ45 is the standard connectors

T1A/E1A 568 6 orange-wit, orange, Green-unit, Blue, Green, Brown-white, Brown

Result :

Born straight drough and Cross - One Eternet cables, were successfully gimped and verified using a cable test . All and 8 pins showed proper connectivity

ne.py > ...

```python
import socket
import struct
sniffer = socket.socket(socket.AF_INET, socket.SOCK_RAW, socket.IPPROTO_IP)
host = socket.gethostbyname(socket.gethostname())
sniffer.bind((host, 0))
sniffer.setsockopt(socket.IPPROTO_IP, socket.IP_HDRINCL, 1)
sniffer.ioctl(socket.SIO_RCVALL, socket.RCVALL_ON)
print("Packet Sniffer Started...\n")
try:
    while True:
        raw_data, addr = sniffer.recvfrom(65535)
        ip_header = raw_data[0:20]
        iph = struct.unpack('!BBHHHBBH4s4s', ip_header)
        version = iph[0] >> 4
        ihl = iph[0] & 0xF
        ttl = iph[5]
        protocol = iph[6]
        source = socket.inet_ntoa(iph[8])
        destination = socket.inet_ntoa(iph[9])
        print("----------------------------------------")
        print("Version :", version)
        print("Header Length:", ihl * 4)
        print("TTL :", ttl)
        print("Protocol :", protocol)
        print("Source IP :", source)
        print("Destination :", destination)
        print("Packet Length:", len(raw_data))
except KeyboardInterrupt:
    sniffer.ioctl(socket.SIO_RCVALL, socket.RCVALL_OFF)
    print("\nStopped")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
Packet Sniffer Started...

----------------------------------------
Version : 4
Header Length: 20
TTL : 1
Protocol : 17
Source IP : 192.168.56.1
Destination : 224.0.0.251
Packet Length: 56
----------------------------------------
Version : 4
Header Length: 20
TTL : 1
Protocol : 17
Source IP : 192.168.56.1
Destination : 224.0.0.251
```

EXP - 4 IMPLEMENT PACKET SNIFFING USING RAW SOCKET IN PYTHON

Aim:
To implement a packet sniffer using raw sockets in python to captive and analyze ethernet, IP, TCP, UDP and ICMP packets

Code:

```
import socket
# Create raw socket
S = socket.socket(Socket.AF.packet, socket, Sock_RAW)
print("packet sniffer started...")
while True:
    raw_data, addr = S.recvfrom(65535)
    print("Packet captured")
    print("raw_data[:50]")
```

Result:
The packet sniffer was successfully implemented using python raw sockets. It captured live network packets and displayed ethernet, IP, TCP, UPP, and ICHP leader information, demonstrating successful packet capture and protocol analysis

gj.py > ...

```python
import subprocess
import platform
print("="*50)
print("Customized Ping Utility")
print("="*50)
host = input("Enter Website or IP Address: ")
system = platform.system()
if system == "Windows":
    command = ["ping", "-n", "4", host]
else:
    command = ["ping", "-c", "4", host]
try:
    result = subprocess.run(command, capture_output=True, text=True)
    print("\nPing Results\n")
    print(result.stdout)
    if result.returncode == 0:
        print("Server Status : Reachable")
    else:
        print("Server Status : Unreachable")
except Exception as e:
    print("Error:", e)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS D:\Computer Network>  d:; cd 'd:\Computer Network'; & 'C:\Users\REC\AppData\Local\Pr
in32-x64\bundled\libs\debugpy\launcher' '54199' '--' 'd:\Computer Network\gj.py'
==================================================
Customized Ping Utility
==================================================
Enter Website or IP Address: google.com

Ping Results


Pinging google.com [142.251.221.142] with 32 bytes of data:
Reply from 142.251.221.142: bytes=32 time=29ms TTL=120
Reply from 142.251.221.142: bytes=32 time=20ms TTL=120
Reply from 142.251.221.142: bytes=32 time=53ms TTL=120
Reply from 142.251.221.142: bytes=32 time=28ms TTL=120

Ping statistics for 142.251.221.142:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 20ms, Maximum = 53ms, Average = 32ms

Server Status : Reachable
```

EXP-5 DEVICE A CUSTOMISED PING COMMAND TO TEST THE SERVER CONNECTIVITY

Aim:
To develope a customized ping command in python dual checks the connectivity between a client and a server by sending ICMP Echo Request packets and displaying due response time and status

Code:

```
impoot subprocess
impoot platFoam
print("="*50)
point("customised ping utility")
point("="*50)
host = input ("Enter website or IP address:")
system = platform.system()
if system == "windows":
    command = ["ping", "-n", "4", host]
else:
    command = ["ping", "-c", "4", host]
try:
    result = subprocess.run(command, capture_output =
True, test = true)
    point("Inping Results\n")
    point (result.stdout)
except Exception as e:
    print ("Error.", e)
```

Result:
The customised ping command was successfully executed and verified the serve connectivity by displaying the ping response and server status (reachable/unreachable).

```python
from ftplib import FTP
from ftplib import all_errors
def anonymous_ftp_scan(host, port=21):
    print("=" * 50)
    print(" ANONYMOUS FTP SCANNER")
    print("=" * 50)
    print(f"Target: {host}")
    print(f"Port  : {port}")
    print()
    ftp = FTP()
    try:
        print("[1] Connecting to FTP server...")
        ftp.connect(host, port, timeout=10)
        print("[+] FTP server is reachable")
        print("[+] Server response:", ftp.getwelcome())
        print("\n[2] Testing anonymous login...")
        ftp.login(user="anonymous", passwd="anonymous@example.com")
        print("[+] Anonymous login SUCCESSFUL")
        print("[!] The FTP server permits anonymous access.")
        print("\n[3] Checking accessible directory...")
        files = ftp.nlst()
        if files:
            print("[+] Directory listing is available:")
            for item in files:
                print(" ", item)
        else:
            print("[-] Directory is empty or listing is unavailable.")
        ftp.quit()
    except all_errors as error:
        print("[-] Anonymous login failed or FTP connection failed.")
        print("[!] Error:", error)
        try:
            ftp.close()
        except:
            pass
target = input("Enter FTP server address: ")
anonymous_ftp_scan(target)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\HDC222041\Downloads\robopath-ai-phase2\robopath-ai> py ftp.py
Enter FTP server address: ftp.cs.brown.edu
==================================================
 ANONYMOUS FTP SCANNER
==================================================
Target: ftp.cs.brown.edu
Port  : 21

[1] Connecting to FTP server...
[+] FTP server is reachable
[+] Server response: 220---------- Welcome to Pure-FTPd [privsep] [TLS] ----------
220-You are user number 1 of 50 allowed.
220-Local time is now 23:24. Server port: 21.
220-IPv6 connections are also welcome on this server.
220 You will be disconnected after 15 minutes of inactivity.

[2] Testing anonymous login...
[+] Anonymous login SUCCESSFUL
[!] The FTP server permits anonymous access.
```

EXP - 6

BUILDING AN ANONYMOUS FTP SCANNER USING PYTHON

Aim:

To develop a python-based anonyms FTP scanner using the built in Ftplit module and determine whether an FTP server permits anonyms access

Procedure:

Open command prompt in windows as terminal in Linux.

Check python Installation.

create two python File

Import the Ftplit module

Write the Scanner program

Run the program

Result:

Building an anonymous FTP scanner Using python has been executed successFully.

```
Last login: Wed Sep 23 17:42:53 on ttys000
(base) happy@Happys-MacBook-Air ~ % cd ~/Desktop/XMLRPC_Calculator
(base) happy@Happys-MacBook-Air XMLRPC_Calculator % python3 client.py
Enter First Number: 20
Enter Second Number: 10

Select Operation
1. Addition
2. Subtraction
3. Multiplication
4. Division
Enter your choice (1-4): 1
Result = 30
(base) happy@Happys-MacBook-Air XMLRPC_Calculator %
```



```
Last login: Wed Sep 23 17:55:57 on ttys000
(base) happy@Happys-MacBook-Air ~ % cd /Users/happy/Experiment8
(base) happy@Happys-MacBook-Air Experiment8 % python3 client.py
Connected to server
Enter message: Hello Server
Server: Message received
Enter message:
```

EXP-7 DEVOLOP A SIMPLE CALCULATOR USING XML-RPC

Aim:

To develop a simple calculator using the XML-RPC (XML Remote procedure Cell) protocol in python

Procedure:

* Install python
* verify python Installation
* creates a project Folder
* write sesver program
* write the client program
* open command prompt
* Navigate to project Folder
* start the XML-RPC sesver
* Open another command prompt
* Run client progoam
* Enter input
* Observe output

Result:

simple calculator using the XML-RPC protocol has successfully executed

```
Last login: Sun Sep 20 20:13:54 on console
(base) happy@Happys-MacBook-Air ~ % python3 --version
Python 3.13.5
(base) happy@Happys-MacBook-Air ~ % mkdir -p ~/Desktop/XMLRPC_Calculator
(base) happy@Happys-MacBook-Air ~ % cd ~/Desktop/XMLRPC_Calculator
(base) happy@Happys-MacBook-Air XMLRPC_Calculator % pwd
/Users/happy/Desktop/XMLRPC_Calculator
(base) happy@Happys-MacBook-Air XMLRPC_Calculator % nano server.py
(base) happy@Happys-MacBook-Air XMLRPC_Calculator % nano client.py
(base) happy@Happys-MacBook-Air XMLRPC_Calculator % python3 server.py
XML-RPC Calculator Server Running on Port 8000...
127.0.0.1 - - [23/Sep/2026 17:46:34] "POST /RPC2 HTTP/1.1" 200 -
```

```
Last login: Wed Sep 23 17:55:50 on ttys000
(base) happy@Happys-MacBook-Air ~ % cd /Users/happy/Experiment8
(base) happy@Happys-MacBook-Air Experiment8 % ls
__pycache__     client.py       server.py
(base) happy@Happys-MacBook-Air Experiment8 % python3 server.py
Waiting for connection...
Connected from: ('127.0.0.1', 55026)
Client: Hello Server
```

EXP - 8

REVERSE SHELL USING TCP SOCKET

Aim:

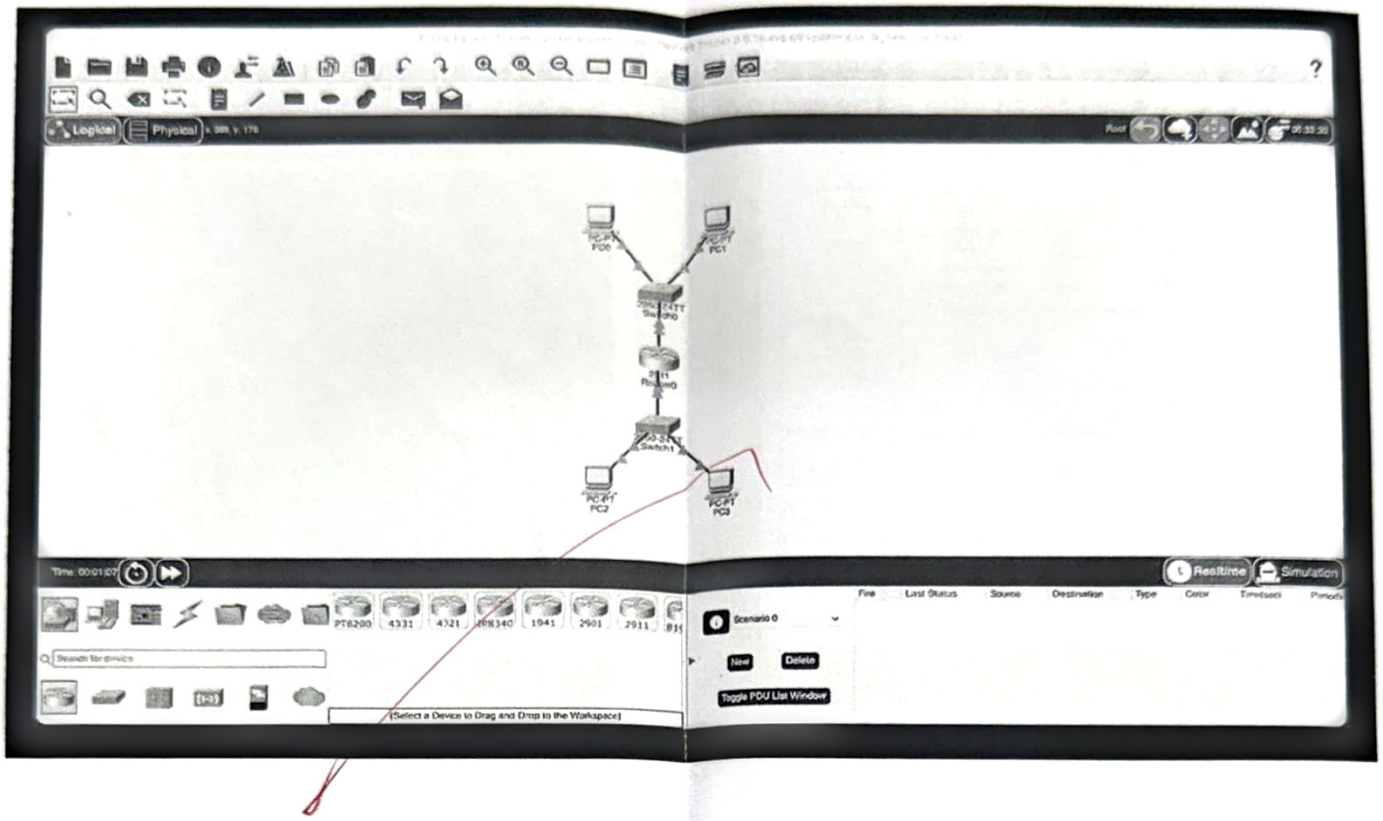
To develop a program to create reverse shell using TCP socket

Procedure:

* Create the server
* Create the client
* Run the server
* Run the client
* Test Multiple Messages

Result:

program to create reverse shell using TCP socket has been executed successfully.


Logical
Physical
Root
PC-PT PC0
PC-PT PC1
2960-24TT Switch0
Router0
2960-24TT Switch1
PC-PT PC2
PC-PT PC3
Time: 00:01:07
Realtime
Simulation
PT6200
4331
4321
ISR4340
1941
2901
2911
Search for devices
[Select a Device to Drag and Drop to the Workspace]
Fire
Last Status
Source
Destination
Type
Color
Time(sec)
Scenario 0
New
Delete
Toggle PDU List Window

EXP-9

SIMPLE NETWORK TOPOLOGY

Aim:

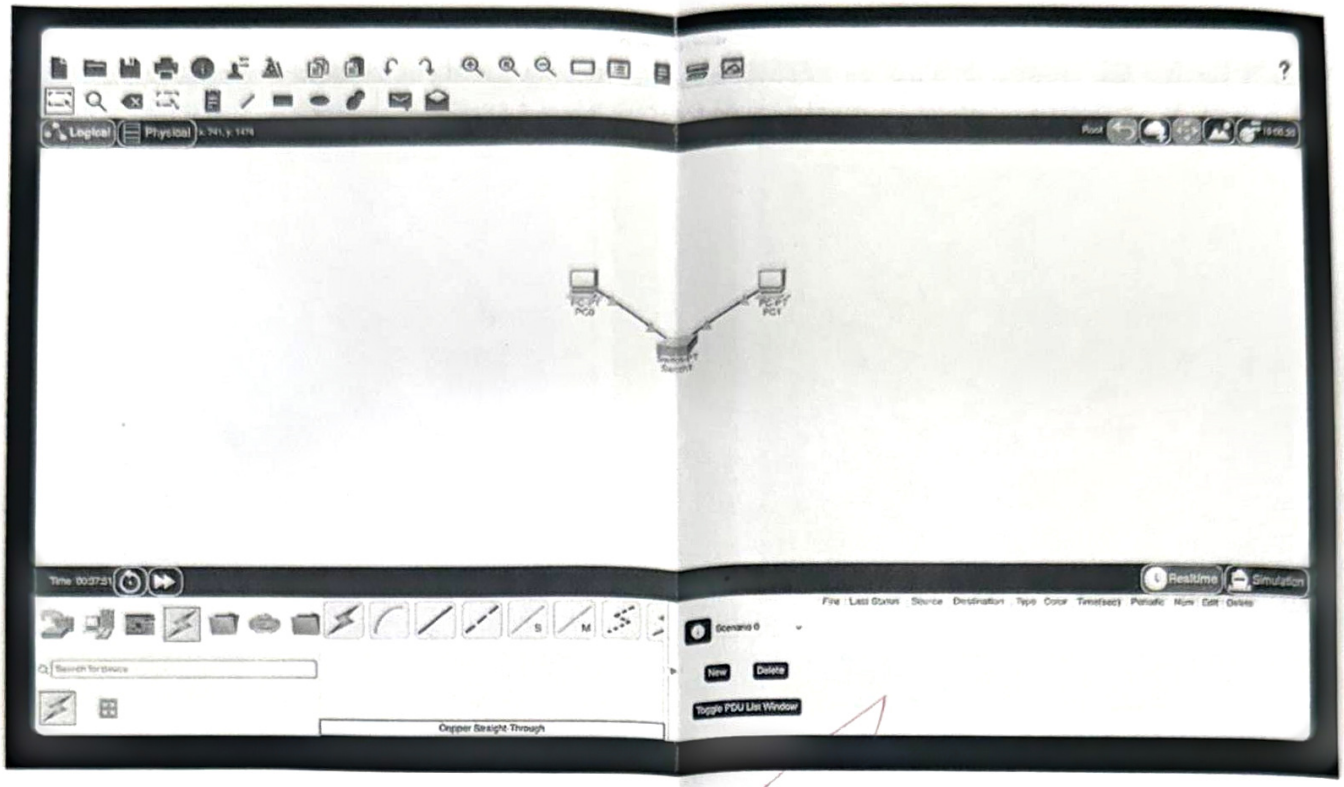
To design and configure a simple network using one router, two switches and four PCs in cisco packet tracker and verify communication between the PCs.

Procedure:

1. open cisco packet Tracker and place 1 router (2911), 2 switches (2960) and 4 PCs
2. connect devices using copper straight-Through cables
3. connect Router G0/0 to switch0 and router G0/1 to switch 1
4. configure router:
   G0/0 → 192.168.1.1
   G0/1 → 192.168.2.1
   subnet mask → 255.255.255.0
   Enable both interfaces using no shutdown
5. configure PCs with the following IP address:
   PC0 → 192.168.1.2
   PC1 → 192.168.1.3
   PC2 → 192.168.2.2
   PC3 → 192.168.2.3
6. set default gateway:
   PC0, PC1 → 192.168.1.1
   PC2, PC3 → 192.168.2.1
7. using ping command to test connectivity between PCs

Result:

Thus, network topology was successfully designed and tested.


Logical
Physical
PC0
PC1
Time: 00:37:31
Realtime
Simulation
Fire
Last Status
Source
Destination
Type
Color
Time(sec)
Periodic
Num
Edit
Delete
Scenario 0
New
Delete
Toggle PDU List Window
Copper Straight-Through

EXP-10

CUSTOMIZE SWITCH WITH NETWORK MODULES USING CISCO PACKET TRACER

Aim:

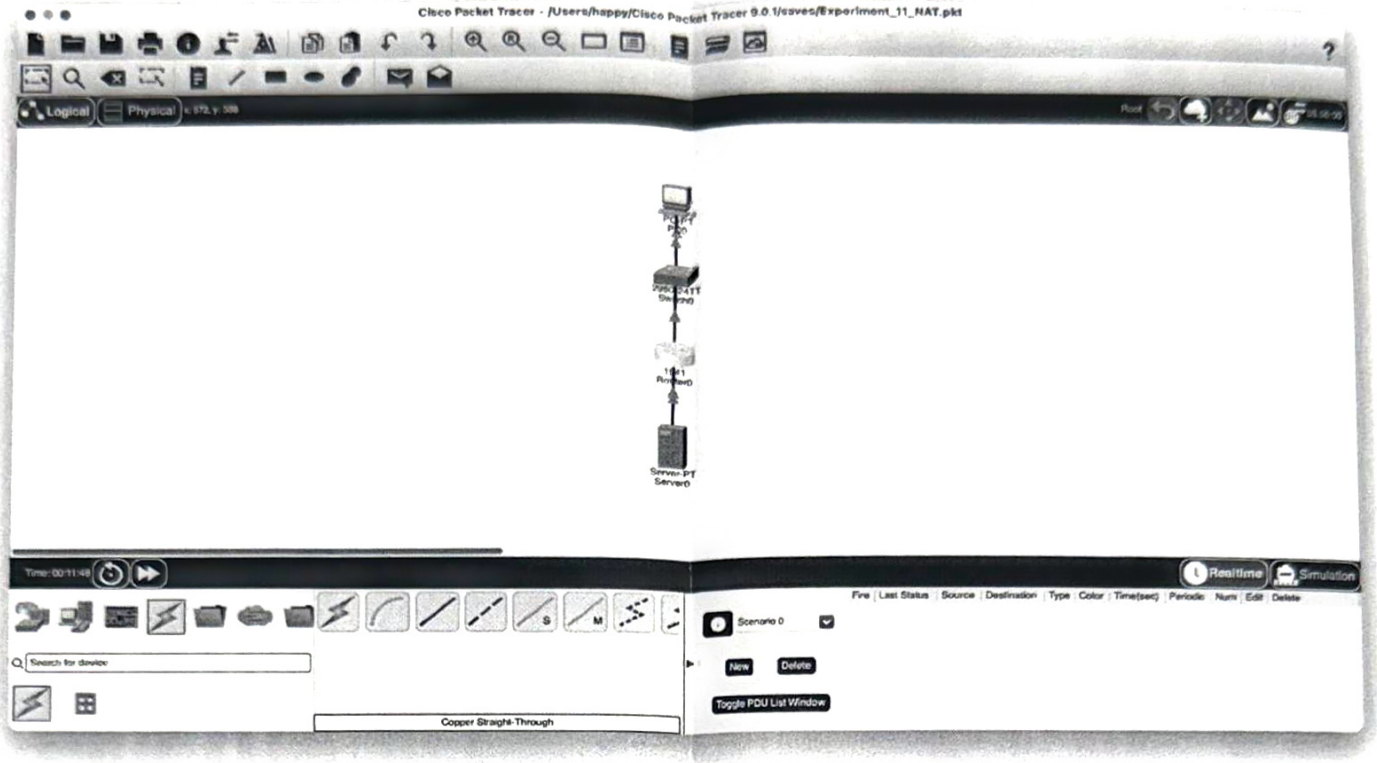
To customize a cisco switch by adding a network Module and verify the changes in switch interfaces.

PROCEDURE:

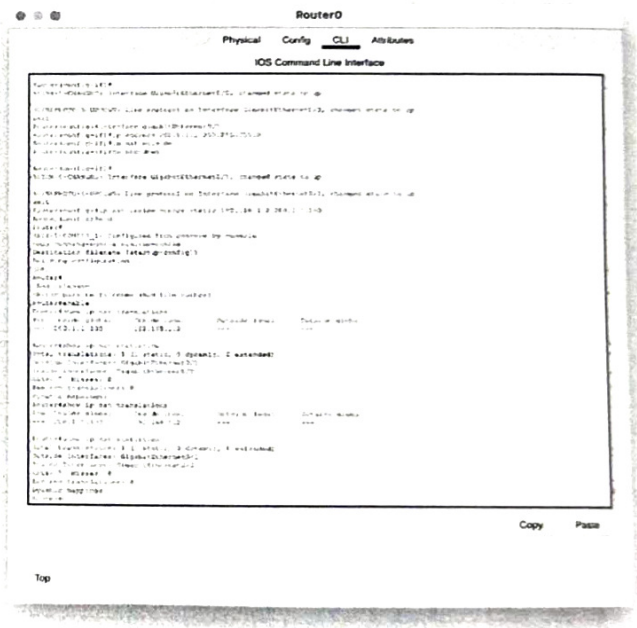
1. open cisco packet Tracer and place a Modular switch
2. open physical tab and powers OFF the switch
3. Insert PT-SWITCH-NM-1CFE module into an empty expansion slot
4. powers ON the switch
5. Open config tab and verify newly added interface
6. connect two PCs to switch using copper straight-through cables
7. configure:
   - PC0 : 192.168.10.2/255.255.255.0
   - PC1 : 192.168.10.3/ 255.255.255.0
8. From PC0, ping PC1 using ping 192.168.10.3

Result:

The switch was successfully customized and communication tested.


Cisco Packet Tracer - /Users/happy/Cisco Packet Tracer 9.0.1/saves/Experiment_11_NAT.pkt
Logical
Physical
Root
Server-PT Server0
Time: 00:11:48
Realtime
Simulation
Fire | Last Status | Source | Destination | Type | Color | Time(sec) | Periodic | Num | Edit | Delete
Scenario 0
New
Delete
Toggle PDU List Window
Search for device
Copper Straight-Through


PC0
Physical
Config
Desktop
Programming
Attributes
Command Prompt
Top


Router0
Physical
Config
CLI
Attributes
IOS Command Line Interface
Copy
Paste
Top

EXP-11 NETWORK ADDRESS TRANSLATION (NAT)

Aim:

To configure and verify NAT using Cisco packet Tracer

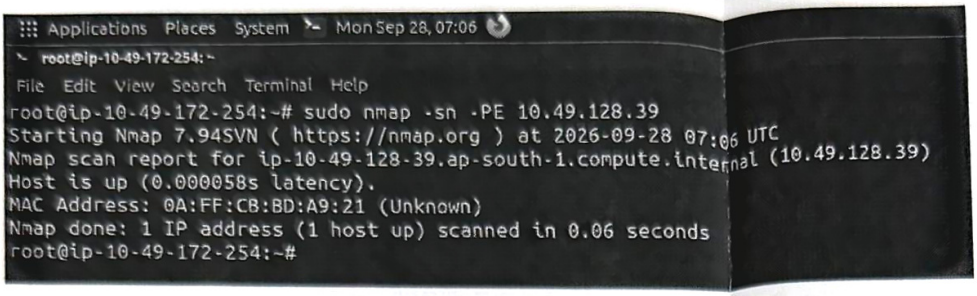
Procedure:

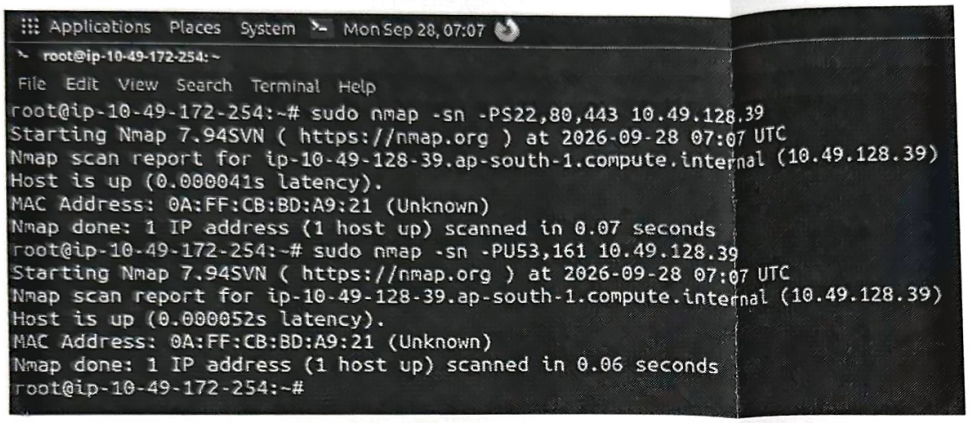
1. Create a topology with 1 routers, 1 switch, 1 PC and 1 server
2. Connect the devices using copper straight-Through cables.
3. Configure router G0/0 as NAT inside with IP 192.168.1.1
4. configure router G0/1 as NAT inside-outside with IP 200.1.1.1
5. Configure static NAT 192.168.1.2 → 200.1.1.100
6. Configure PC and server IP addresses
7. Ping 200.1.1.2 From PC0
8. Verify using show ip nat translation and show IP nat statistics.

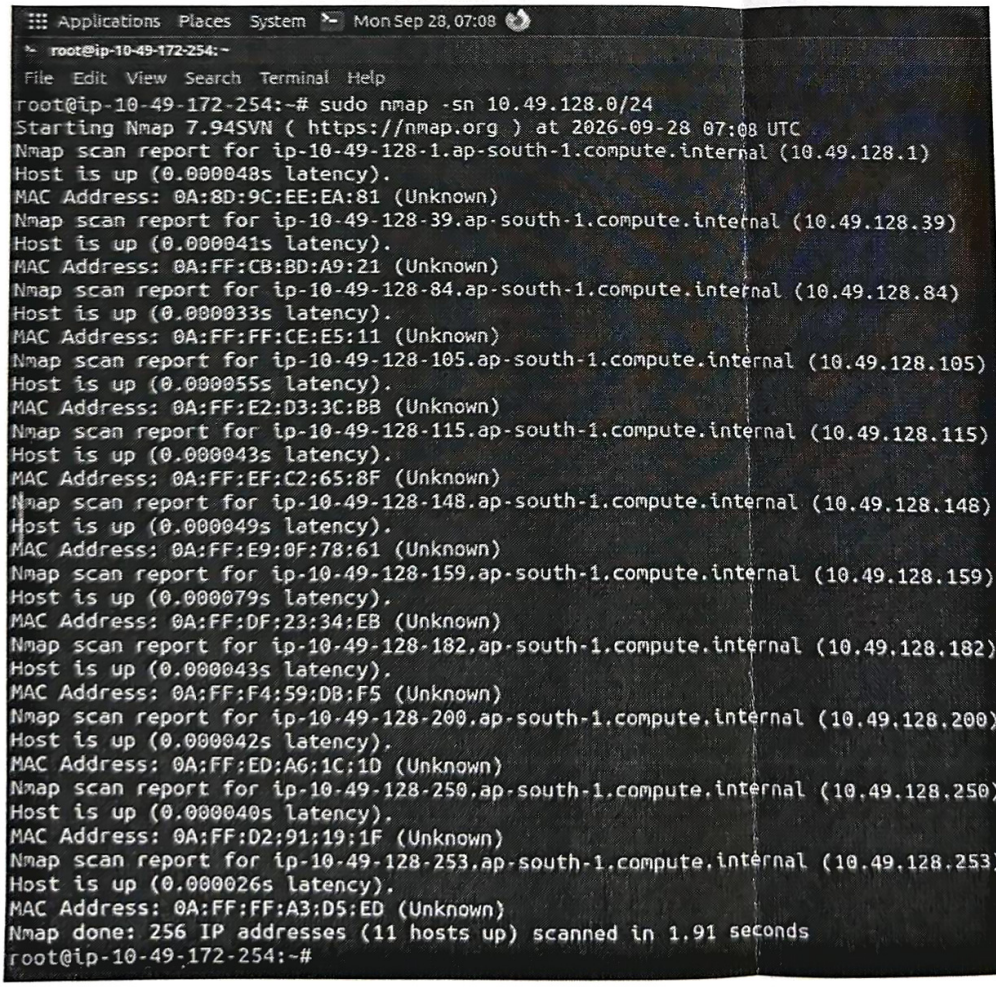
Result:

NAT was successfully configured and verified.
Ping was successful with 0% packet loss


Applications Places System Mon Sep 28, 07:06
root@ip-10-49-172-254: ~
File Edit View Search Terminal Help
root@ip-10-49-172-254:~# nmap --version
Nmap version 7.94SVN (https://nmap.org)
Platform: x86_64-pc-linux-gnu
Compiled with: liblua-5.4.6 openssl-3.0.13 libssh2-1.11.0 libz-1.3 libpcre2-10.42 libpcap-1.10.4 nmap-libdnet-1.12 ipv6
Compiled without:
Available nsock engines: epoll poll select
root@ip-10-49-172-254:~#


Applications Places System Mon Sep 28, 07:06
root@ip-10-49-172-254: ~
File Edit View Search Terminal Help
root@ip-10-49-172-254:~# sudo nmap -sn -PE 10.49.128.39
Starting Nmap 7.94SVN (https://nmap.org) at 2026-09-28 07:06 UTC
Nmap scan report for ip-10-49-128-39.ap-south-1.compute.internal (10.49.128.39)
Host is up (0.000058s latency).
MAC Address: 0A:FF:CB:BD:A9:21 (Unknown)
Nmap done: 1 IP address (1 host up) scanned in 0.06 seconds
root@ip-10-49-172-254:~#


Applications Places System Mon Sep 28, 07:07
root@ip-10-49-172-254: ~
File Edit View Search Terminal Help
root@ip-10-49-172-254:~# sudo nmap -sn -PS22,80,443 10.49.128.39
Starting Nmap 7.94SVN (https://nmap.org) at 2026-09-28 07:07 UTC
Nmap scan report for ip-10-49-128-39.ap-south-1.compute.internal (10.49.128.39)
Host is up (0.000041s latency).
MAC Address: 0A:FF:CB:BD:A9:21 (Unknown)
Nmap done: 1 IP address (1 host up) scanned in 0.07 seconds
root@ip-10-49-172-254:~# sudo nmap -sn -PU53,161 10.49.128.39
Starting Nmap 7.94SVN (https://nmap.org) at 2026-09-28 07:07 UTC
Nmap scan report for ip-10-49-128-39.ap-south-1.compute.internal (10.49.128.39)
Host is up (0.000052s latency).
MAC Address: 0A:FF:CB:BD:A9:21 (Unknown)
Nmap done: 1 IP address (1 host up) scanned in 0.06 seconds
root@ip-10-49-172-254:~#


Applications Places System Mon Sep 28, 07:08
root@ip-10-49-172-254: ~
File Edit View Search Terminal Help
root@ip-10-49-172-254:~# sudo nmap -sn 10.49.128.0/24
Starting Nmap 7.94SVN (https://nmap.org) at 2026-09-28 07:08 UTC
Nmap scan report for ip-10-49-128-1.ap-south-1.compute.internal (10.49.128.1)
Host is up (0.000048s latency).
MAC Address: 0A:8D:9C:EE:EA:81 (Unknown)
Nmap scan report for ip-10-49-128-39.ap-south-1.compute.internal (10.49.128.39)
Host is up (0.000041s latency).
MAC Address: 0A:FF:CB:BD:A9:21 (Unknown)
Nmap scan report for ip-10-49-128-84.ap-south-1.compute.internal (10.49.128.84)
Host is up (0.000033s latency).
MAC Address: 0A:FF:FF:CE:E5:11 (Unknown)
Nmap scan report for ip-10-49-128-105.ap-south-1.compute.internal (10.49.128.105)
Host is up (0.000055s latency).
MAC Address: 0A:FF:E2:D3:3C:BB (Unknown)
Nmap scan report for ip-10-49-128-115.ap-south-1.compute.internal (10.49.128.115)
Host is up (0.000043s latency).
MAC Address: 0A:FF:EF:C2:65:8F (Unknown)
Nmap scan report for ip-10-49-128-148.ap-south-1.compute.internal (10.49.128.148)
Host is up (0.000049s latency).
MAC Address: 0A:FF:E9:0F:78:61 (Unknown)
Nmap scan report for ip-10-49-128-159.ap-south-1.compute.internal (10.49.128.159)
Host is up (0.000079s latency).
MAC Address: 0A:FF:DF:23:34:EB (Unknown)
Nmap scan report for ip-10-49-128-182.ap-south-1.compute.internal (10.49.128.182)
Host is up (0.000043s latency).
MAC Address: 0A:FF:F4:59:DB:F5 (Unknown)
Nmap scan report for ip-10-49-128-200.ap-south-1.compute.internal (10.49.128.200)
Host is up (0.000042s latency).
MAC Address: 0A:FF:ED:A6:1C:1D (Unknown)
Nmap scan report for ip-10-49-128-250.ap-south-1.compute.internal (10.49.128.250)
Host is up (0.000040s latency).
MAC Address: 0A:FF:D2:91:19:1F (Unknown)
Nmap scan report for ip-10-49-128-253.ap-south-1.compute.internal (10.49.128.253)
Host is up (0.000026s latency).
MAC Address: 0A:FF:FF:A3:D5:ED (Unknown)
Nmap done: 256 IP addresses (11 hosts up) scanned in 1.91 seconds
root@ip-10-49-172-254:~#

EXP-12

NMAP LIVE HOST DISCOVERY

Aim:

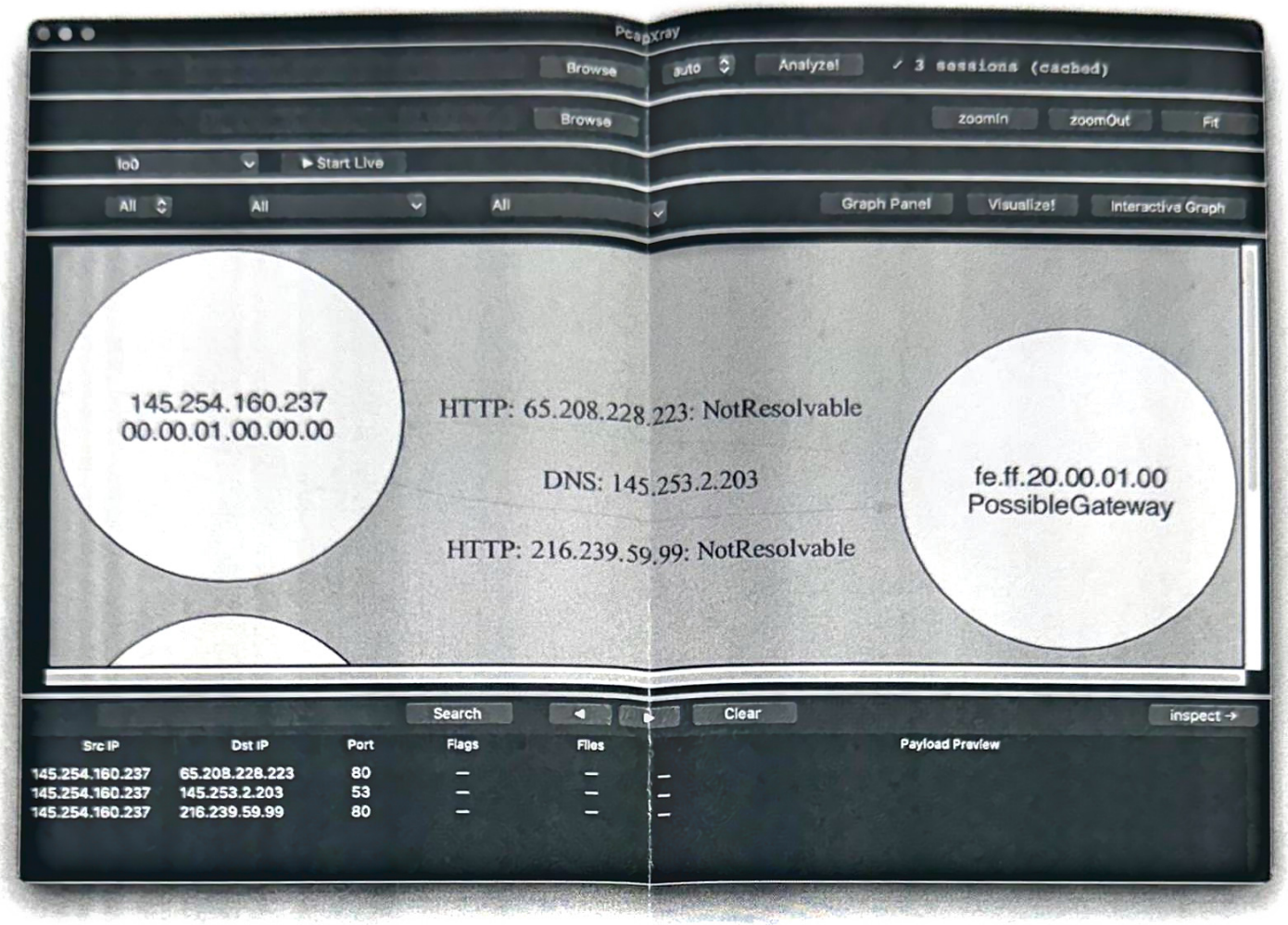
To discover live hosts in a network using nmap through ARP, ICMP, TCP, UDP ping scans

Procedure:

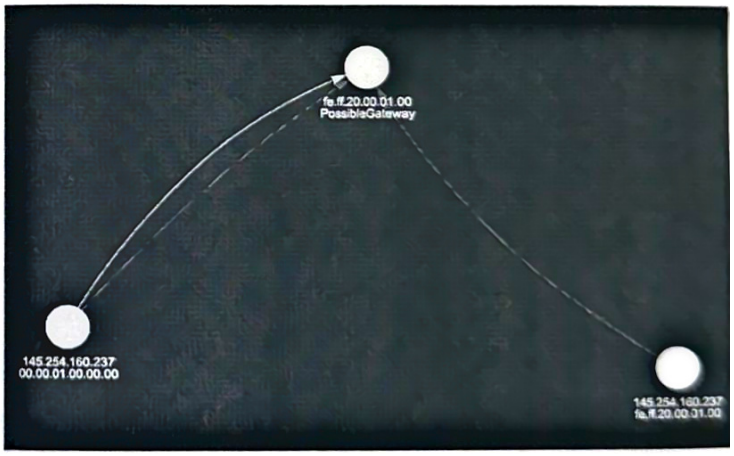
1. Open TryHackMe and start the Nmap Basic ports scan Lab machine and AttackBox
2. Open AttackBox terminal and verify Nmap using nmap --version
3. perform ICMP scan
4. perform ARP scan
5. perform TCP scan
6. perform UCP scan
7. Scan the subnet to identify hosts

Result:

Nmap was successfully used to discover live hosts using different host-discovery techniques.


PcapXray
Browse
auto
Analyze!
✓ 3 sessions (cached)
Browse
zoomIn
zoomOut
Fit
lo0
▶ Start Live
All
All
All
Graph Panel
Visualize!
Interactive Graph
145.254.160.237
00.00.01.00.00.00
HTTP: 65.208.228.223: NotResolvable
DNS: 145.253.2.203
HTTP: 216.239.59.99: NotResolvable
fe.ff.20.00.01.00
PossibleGateway
Search
Clear
inspect →
Src IP
Dst IP
Port
Flags
Files
Payload Preview
145.254.160.237 65.208.228.223 80
145.254.160.237 145.253.2.203 53
145.254.160.237 216.239.59.99 80


Session: 145.254.160.237/65.208.228.223/80
Forward (0 entries)
Reverse (15 entries)
Close


fe.ff.20.00.01.00
PossibleGateway
145.254.160.237
00.00.01.00.00.00

EXP-13 NETWORK FORENSICS USING PCAPXRAY

Aim:

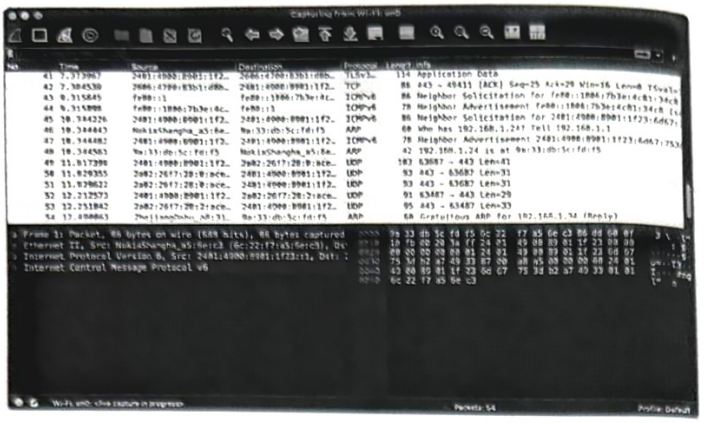
To analyse a pcap file using pcapxray and identify network communication and security indicators.

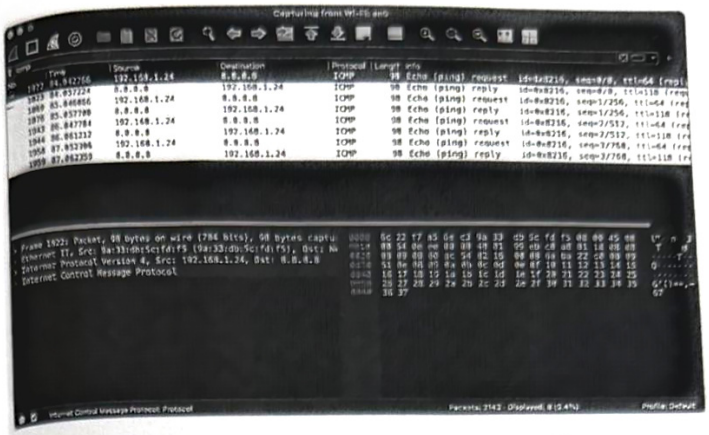
Procedure:

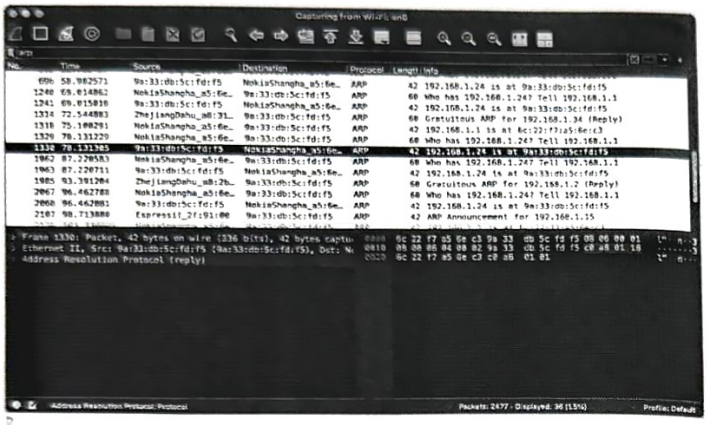
1. Open pcapxray and load the http cap file
2. Analyze the pcap and observe network sessions
3. Inspect a session and check its details
4. Generate the network visualization and interactive graph.

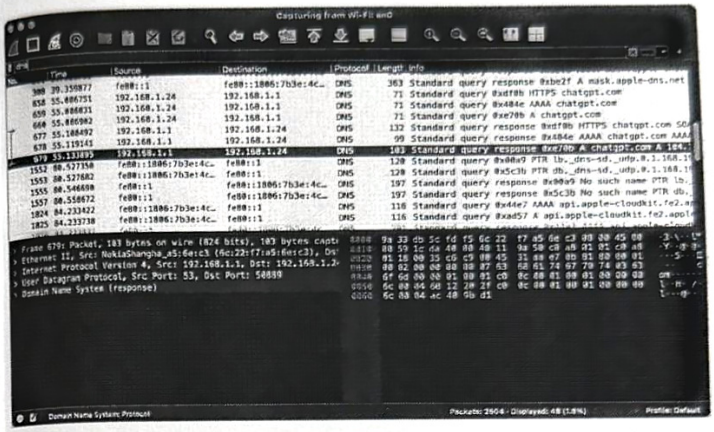
Result:

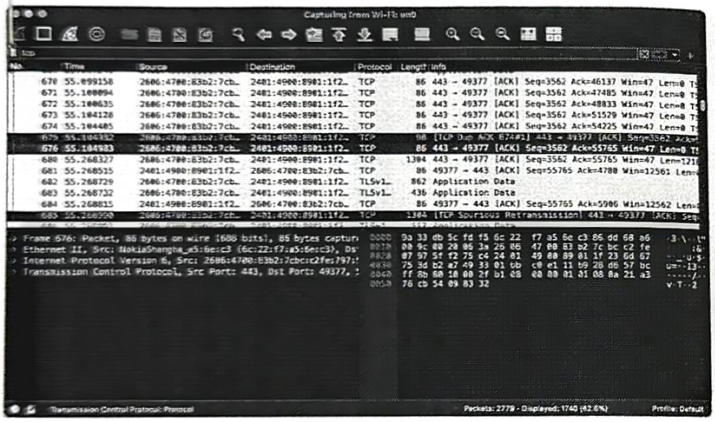
The pcap File was successfully analysed using pcapxray and the network sessions, sessions details and interactive graph were successfully obtained.

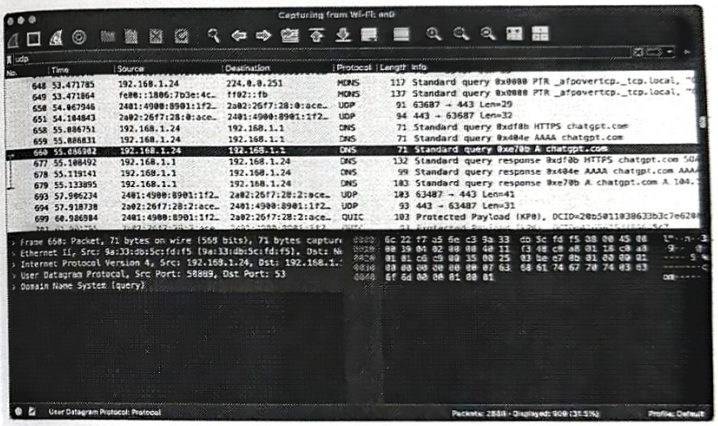


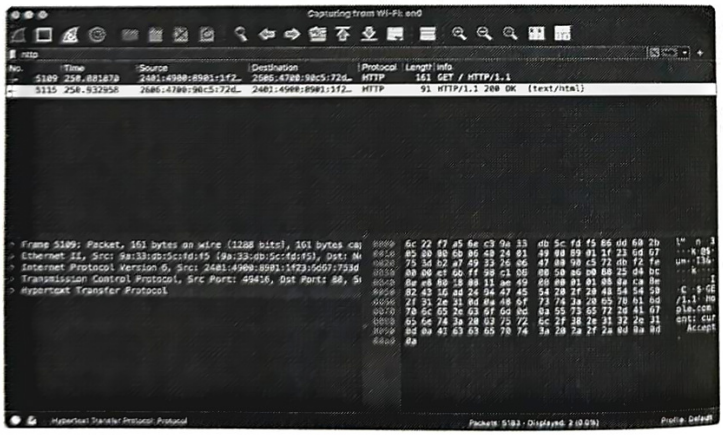


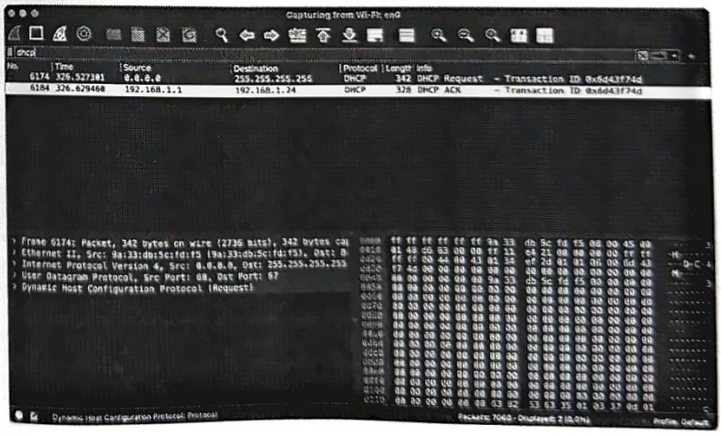












EXP- 14 | TO CAPTURE, ANALYSE NETWORK TRAFFIC USING WIRESHARK

Aim:

TO capture and analyse TCP, UDP, IP, HTTP, ARP, DHCP, ICMP and DNS using Wireshark

Procedure:

Open wireshark

Select active wi-Fi interface

Start capture

General network traffic

Apply protocol Filters

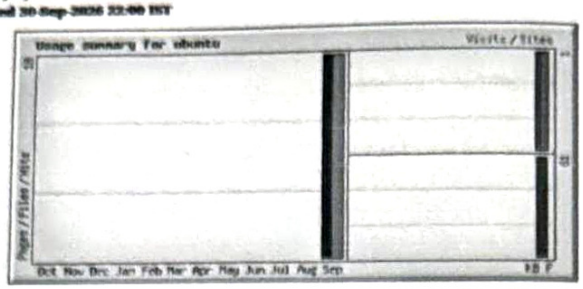
Analyse packets

Result:

Network traffic was successfully captured and analyzed using wireshark.

Usage statistics for ubuntu

Summary by Month
Generated 30-Sep-2026 22:00 IST



Summary by Month										
Month	Daily Avg				Monthly Totals					
	Hits	Files	Pages	Visits	Sites	kB F	Visits	Pages	Files	Hits
Sep 2026	18	18	18	1	1	61	1	18	18	18
Totals						61	1	18	18	18

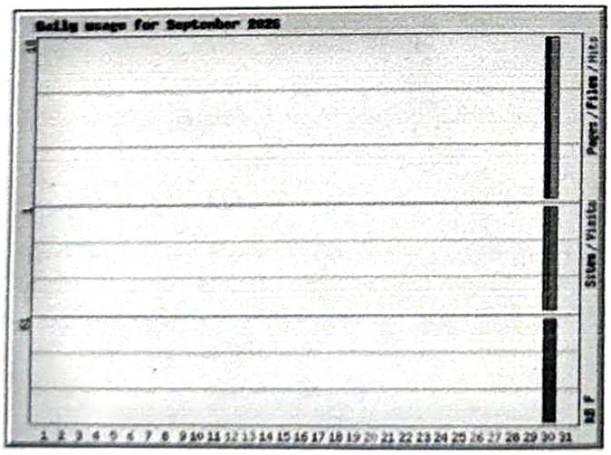
Generated by Webalizer Version 2.23

Usage statistics for ubuntu

Summary Period: September 2026
Generated 30-Sep-2026 22:00 IST

[Daily Statistics] [Hourly Statistics] [URLs] [Entry] [Exit] [Sites] [Referrers] [Search] [Agents] [Countries]

Monthly Statistics for September 2026		
Total Hits		18
Total Files		18
Total Pages		18
Total Visits		1
Total kB Files		61
Total Unique Sites		1
Total Unique URLs		1
Total Unique Referrers		1
Total Unique User Agents		1
	Avg	Max
Hits per Hour	0	18
Hits per Day	18	18
Files per Day	18	18
Pages per Day	18	18
Sites per Day	1	1
Visits per Day	1	1
kB Files per Day	61	61
Hits by Response Code		
Code 200 - OK	100.00%	18



Daily Statistics for September 2026												
Day	Hits		Files		Pages		Visits		Sites		kB F	
30	18	100.00%	18	100.00%	18	100.00%	1	100.00%	1	100.00%	61	100.00%

Hourly Statistics for September 2026												
Hour	Hits			Files			Pages			kB F		
	Avg	Total		Avg	Total		Avg	Total		Avg	Total	
0	0	0	0.00%	0	0	0.00%	0	0	0.00%	0	0	0.00%
1	0	0	0.00%	0	0	0.00%	0	0	0.00%	0	0	0.00%
2	0	0	0.00%	0	0	0.00%	0	0	0.00%	0	0	0.00%
3	0	0	0.00%	0	0	0.00%	0	0	0.00%	0	0	0.00%
4	0	0	0.00%	0	0	0.00%	0	0	0.00%	0	0	0.00%
5	0	0	0.00%	0	0	0.00%	0	0	0.00%	0	0	0.00%
6	0	0	0.00%	0	0	0.00%	0	0	0.00%	0	0	0.00%
7	0	0	0.00%	0	0	0.00%	0	0	0.00%	0	0	0.00%
8	0	0	0.00%	0	0	0.00%	0	0	0.00%	0	0	0.00%
9	0	0	0.00%	0	0	0.00%	0	0	0.00%	0	0	0.00%
10	0	0	0.00%	0	0	0.00%	0	0	0.00%	0	0	0.00%
11	0	0	0.00%	0	0	0.00%	0	0	0.00%	0	0	0.00%
12	0	0	0.00%	0	0	0.00%	0	0	0.00%	0	0	0.00%
13	0	0	0.00%	0	0	0.00%	0	0	0.00%	0	0	0.00%
14	0	0	0.00%	0	0	0.00%	0	0	0.00%	0	0	0.00%
15	0	0	0.00%	0	0	0.00%	0	0	0.00%	0	0	0.00%
16	0	0	0.00%	0	0	0.00%	0	0	0.00%	0	0	0.00%
17	0	0	0.00%	0	0	0.00%	0	0	0.00%	0	0	0.00%
18	0	0	0.00%	0	0	0.00%	0	0	0.00%	0	0	0.00%
19	0	0	0.00%	0	0	0.00%	0	0	0.00%	0	0	0.00%
20	0	0	0.00%	0	0	0.00%	0	0	0.00%	0	0	0.00%
21	18	18	100.00%	18	18	100.00%	18	18	100.00%	61	61	100.00%
22	0	0	0.00%	0	0	0.00%	0	0	0.00%	0	0	0.00%
23	0	0	0.00%	0	0	0.00%	0	0	0.00%	0	0	0.00%

Top 1 of 1 Total URLs					
#	Hits		kB F		URL
1	18	100.00%	61	100.00%	/

Top 1 of 1 Total URLs By kB F					
#	Hits		kB F		URL
1	18	100.00%	61	100.00%	/

Top 1 of 1 Total Entry Pages					
#	Hits		Visits		URL
1	18	100.00%	1	100.00%	/

Top 1 of 1 Total Sites									
#	Hits		Files		kB F		Visits		Hostname
1	18	100.00%	18	100.00%	61	100.00%	1	100.00%	127.0.0.1

Top 1 of 1 Total Sites By kB F									
#	Hits		Files		kB F		Visits		Hostname
1	18	100.00%	18	100.00%	61	100.00%	1	100.00%	127.0.0.1

Top 1 of 1 Total Referrers			
#	Hits		Referrer
1	18	100.00%	- (Direct Request)

Top 1 of 1 Total User Agents			
#	Hits		User Agent
1	18	100.00%	Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:133.0) Gecko/20100101 Firefox/133.0



Top 1 of 1 Total Countries							
#	Hits		Files		kB F		Country
1	18	100.00%	18	100.00%	61	100.00%	Unresolved/Unknown

EXP-15 WEBALIZER

Aim:

To install, configure and use webalizer to analyse web server log files

Procedure:

Install Apache and webalizer
Generate web traffic
Configure the apaches access log
Generate webalizer report
View report in browser

Result:

Webalizer successfully analysed apache log file and generated statistical reports